

Dispensa · Modulo 7

1. Ripasso: liste, while e funzioni

Liste

Una lista è una struttura dati di Python: una sequenza **ordinata** di valori. Ordinata significa che ogni elemento ha una posizione precisa (0, 1, 2, ...) e ha senso chiedersi quale elemento viene prima e quale dopo.

È una **struttura** dati e non un **dato primario** perché ogni lista è composta da più elementi al suo interno, ciascuno dei quali è a sua volta un dato. Un dato primario — come un intero, un float o una stringa — rappresenta un singolo valore atomico, indivisibile. Una lista invece ha una **struttura interna**: organizza più valori in sequenza, li rende accessibili tramite indice e mette a disposizione operazioni specifiche per aggiungere, modificare o scorrere il contenuto.

Le liste sono **mutabili**: dopo la creazione, il loro contenuto può essere modificato. Si inizializzano con le parentesi quadre e si accede agli elementi ancora con le parentesi quadre:

```
numeri = [10, 20, 30]
print(numeri[0])    # 10
numeri[1] = 99
print(numeri)       # [10, 99, 30]
```

Per aggiungere un elemento in fondo si usa `append()`:

```
numeri.append(40)
print(numeri)      # [10, 99, 30, 40]
```

Tipi mutabili e immutabili

Tipo	Mutabile?	Conseguenza
list	Sì	il contenuto può cambiare dopo la creazione (<code>lista[0] = 99</code> , <code>lista.append(x)</code>)
int, float, str, bool	No	non si può modificare il valore — si può solo crearne uno nuovo

La distinzione è importante nelle funzioni: riassegnare un intero dentro una funzione non cambia mai la variabile esterna; modificare una lista in-place sì.

Il ciclo while

La struttura generale è:

```
while condizione_booleana:
    istruzioni
```

Il comportamento:

1. si valuta la condizione;
2. se è vera, si esegue il blocco;
3. poi la condizione viene rivalutata;
4. il ciclo continua finché la condizione rimane vera;
5. quando diventa falsa, il programma prosegue con la prima istruzione fuori dal blocco.

Se la condizione non diventa mai falsa, il ciclo non termina mai.

Funzioni: definizione e chiamata

Bisogna distinguere due momenti:

- la **definizione** introduce la funzione con **def**, il nome e la lista dei parametri — non esegue nulla, crea solo un “modello”;
- la **chiamata** esegue effettivamente il codice.

```
def doppio(n):  
    return n * 2
```

```
risultato = doppio(5)    # chiamata  
print(risultato)         # 10
```

Una funzione può essere definita una volta e chiamata molte volte, con argomenti diversi.

Espressioni e istruzioni

In Python bisogna distinguere due tipi di costrutti:

- un'**espressione** è qualcosa che produce un valore — può comparire a destra di un **=**, dentro una condizione, come argomento di un'altra funzione;
- un'**istruzione** è un'azione che il programma esegue, ma non produce un valore — si usa da sola, non si può assegnare a una variabile.

```
3 + 2          # espressione: produce 5  
doppio(5)      # espressione: produce 10 (se doppio restituisce un valore)  
print("ciao")  # istruzione: esegue un'azione, non produce nulla di utile
```

Questo ha una conseguenza diretta sulle chiamate di funzione:

- la chiamata a una **funzione pura** (con **return**) è un'**espressione** — il suo valore si può usare, assegnare, passare ad altri:

```
risultato = doppio(5)    # ha senso: doppio(5) vale 10  
print(doppio(5))        # ha senso: si stampa 10
```

- la chiamata a una **funzione void** (senza **return**) è un'**istruzione** — si chiama e basta, assegnarla non ha senso:

```

lista.append(4)           # ha senso: esegue l'azione
x = lista.append(4)       # non ha senso: x varrà None

```

Funzione pura vs. funzione void

	Funzione pura	Funzione void
Restituisce un valore?	sì (return)	no
Modifica il suo parametro?	no	spesso sì
Come si usa il risultato?	risultato = f(x) — la chiamata è un' espressione	si chiama e basta — la chiamata è un' istruzione

Scope

Le variabili definite dentro una funzione sono **locali**: non esistono fuori.

```

def doppio(n):
    n = n * 2      # n locale - non tocca la variabile esterna
    return n

x = 3
y = doppio(x)
print(x) # 3 - x non è cambiato
print(y) # 6

```

Regola pratica: riassegnare un parametro dentro la funzione non cambia mai la variabile esterna, a prescindere dal tipo.

Aliasing

Quando si assegna una lista a un secondo nome, i due nomi puntano alla **stessa lista** in memoria.

```

a = [1, 2, 3]
b = a      # b non è una copia: b è un alias di a

b[0] = 99
print(a)   # [99, 2, 3] - anche a è cambiata

```

Questo spiega perché una funzione void può modificare una lista passata come argomento: dentro la funzione il parametro è un alias della lista originale.

2. Testing ragionato

Quando un programma produce la risposta attesa su un input, non basta concludere che sia corretto. Bisogna chiedersi:

- ha funzionato per il motivo giusto?
- oppure ha restituito il risultato corretto solo per caso?

Il valore dei controesempi

Un **controesempio** è un input per cui:

- noi ci aspettiamo un certo risultato;
- il programma produce un risultato diverso.

Trovare un controesempio è una prova concreta che il programma non funziona. Il problema opposto è più subdolo: il programma è sbagliato, ma sull'input scelto restituisce comunque il valore giusto — l'errore resta nascosto.

Scegliere input piccoli

Per analizzare il comportamento di un programma conviene partire da input **minimi e semplici**:

- evitare liste lunghe all'inizio;
- usare liste di 2 o 3 elementi;
- scrivere lo stato delle variabili passo per passo.

Questo rende più facile capire come si muovono gli indici, quali rami vengono eseguiti e dove si trova l'errore.

Per trovare un errore logico servono input “furbi”: casi in cui la risposta giusta è **sì** e casi in cui è **no**, e confrontiamo quello che ci aspettiamo con quello che otteniamo.

3. Tracciamento passo per passo

Il tracciamento è lo strumento principale per analizzare l'esecuzione di un programma. Consiste nello scrivere esplicitamente il valore di ogni variabile dopo ogni istruzione.

Formato della tabella

Passo	Codice sorgente	Stato del programma	Output
-------	-----------------	---------------------	--------

Non è necessario un formato grafico preciso. L'importante è che la tabella permetta di seguire con chiarezza l'evoluzione dello stato.

Tracciamento con funzioni e aliasing

```
def raddoppia(lista):  
    i = 0  
    while i < len(lista):  
        lista[i] = lista[i] * 2  
        i = i + 1  
  
valori = [1, 3, 5]  
raddoppia(valori)  
print(valori)
```

Passo	Codice sorgente	Stato globale	Stato funzione	Output
1	valori = [1, 3, 5]	valori=[1,3,5]	—	
2	raddoppia(valori)	valori=[1,3,5]	lista→valori, i=0	
3	lista[0] = 1*2	valori=[2,3,5]	lista→valori, i=0	
4	i = i + 1	valori=[2,3,5]	lista→valori, i=1	
5	lista[1] = 3*2	valori=[2,6,5]	lista→valori, i=1	
6	i = i + 1	valori=[2,6,5]	lista→valori, i=2	
7	lista[2] = 5*2	valori=[2,6,10]	lista→valori, i=2	
8	i = i + 1	valori=[2,6,10]	lista→valori, i=3	
9	fine raddoppia	valori=[2,6,10]	—	
10	print(valori)	valori=[2,6,10]	—	[2, 6, 10]

Cosa osservare: `lista` e `valori` puntano alla stessa lista in memoria. Le modifiche fatte dentro la funzione (in-place) sono visibili fuori.

4. Trova l'errore

Questa sezione raccoglie tre errori tipici su cicli annidati. Per ciascuno, il metodo è lo stesso: prima verificare se il programma termina, poi se produce il risultato corretto.

Esempio 1 — ciclo interno che non termina

Il programma vuole stampare se in `lista` c'è almeno un elemento ripetuto oppure no.

```

lista = [1, 2, 3, 1, 4]

trovato = False
i = 0
while i < len(lista):
    j = i + 1
    while j < len(lista):
        if lista[j] == lista[i]:
            trovato = True
        i = i + 1

if trovato:
    print("c'è almeno un elemento ripetuto")
else:
    print("nessun elemento ripetuto")

```

Proviamo su `lista = [1, 2, 1]` per capire come avanza l'esecuzione:

Riga	i	j	trovato	i < len	j < len	lista[j] == lista[i]
...	0	—	False			
...	0	1	False		1<3? Sì	
...	0	1	False			lista[1]=2 == lista[0]=1? No
...	0	1	False		1<3? Sì	
...	0	1	False			lista[1]=2 == lista[0]=1? No
...	...	...	...		...	...

`j` non cambia mai: il ciclo interno non termina mai.

Errore: dentro il ciclo interno manca `j = j + 1`. La guardia `j < len(lista)` non diventa mai falsa.

Codice corretto:

```

while j < len(lista):
    if lista[j] == lista[i]:
        trovato = True
    j = j + 1    # ← aggiunto

```

Esempio 2 — variabile di flag reinizializzata dentro il ciclo

Il programma vuole stampare se in `lista` c'è almeno un elemento ripetuto oppure no.

```

lista = [1, 2, 3, 1, 4]

i = 0
while i < len(lista):
    trovato = False          # ← posizione sospetta
    j = i + 1
    while j < len(lista):
        if lista[j] == lista[i]:
            trovato = True
        j = j + 1
    i = i + 1

if trovato:
    print("c'è almeno un elemento ripetuto")
else:
    print("nessun elemento ripetuto")

```

Input furbi: `lista = [1, 1]` (risposta attesa: *c'è un elemento ripetuto*) e `lista = [1, 2]` (risposta attesa: *nessun elemento ripetuto*).

Con `lista = [1, 1]`:

Riga	i	j	trovato	note
...	0	—	False	reinizializzato
...	0	1	True	trovato!
...	1	—	False ← reset!	reinizializzato alla seconda iterazione esterna
...	1	2	False	j parte da 2, non confronta nulla

Fine ciclo: `trovato = False` → stampa “nessun elemento ripetuto” — sbagliato!

`[1, 2]` non rivelerebbe nulla perché la risposta corretta era già “no”. È `[1, 1]` che smonta il programma.

Errore: `trovato = False` è dentro il ciclo esterno, quindi viene reimpostato a `False` a ogni iterazione. Il valore `True` guadagnato viene cancellato alla passata successiva.

Codice corretto: spostare `trovato = False` **prima** del ciclo esterno.

Esempio 3 — guardia del ciclo interno troppo larga

Il programma vuole costruire la lista che, per ogni elemento di `lista`, conta quanti elementi **uguali a lui lo precedono**.

```
lista = [1, 2, 1, 3, 1]

contatori = []
i = 0
while i < len(lista):
    contatore = 0
    j = 0
    while j < len(lista):    # ← guardia sospetta
        if lista[j] == lista[i]:
            contatore = contatore + 1
        j = j + 1
    contatori.append(contatore)
    i = i + 1

print(contatori)
```

Input furbo: `lista = [1, 1]`. Risposta attesa: `[0, 1]` — il primo 1 non ha precedenti uguali; il secondo ne ha uno.

Tracciamo la prima iterazione esterna (`i=0`):

i	j	contatore	lista[j] == lista[i]
0	0	0 → 1	lista[0]=1 == lista[0]=1? Sì
0	1	1 → 2	lista[1]=1 == lista[0]=1? Sì

Il primo elemento ottiene `contatore = 2`, ma la risposta attesa era 0. Il ciclo interno ha contato **tutti** gli 1, non solo quelli che vengono prima di `i`.

Errore: la guardia del ciclo interno è `j < len(lista)` invece di `j < i`. Così `j` scorre tutta la lista, contando anche gli elementi successivi e perfino `lista[i]` stesso.

Codice corretto:

```
while j < i:    # ← solo gli elementi prima di i
    if lista[j] == lista[i]:
        contatore = contatore + 1
    j = j + 1
```


5. Errori frequenti nei cicli while

Indentazione sbagliata

In Python l'indentazione non è un dettaglio grafico: **definisce il flusso del programma**. Se un blocco è indentato male:

- un `else` può non riferirsi all'`if` giusto;
- un `print` può finire dentro o fuori dal ciclo in modo scorretto;
- il flusso complessivo diventa ambiguo o direttamente sbagliato.

Errore tipico: un `while` scritto allo stesso livello di un `if` quando dovrebbe appartenergli:

```
# SBAGLIATO
if condizione:
    x = 0
while x < 10:      # questo while è fuori dall'if!
    x = x + 1

# CORRETTO
if condizione:
    x = 0
    while x < 10:  # questo while è dentro l'if
        x = x + 1
```

Condizione di arresto: > contro >=

Un errore frequente riguarda la scelta tra `>` e `>=` nella condizione del `while`. Usare `>=` quando serve `>` fa eseguire il ciclo un passo in più, producendo un risultato sovrastimato di 1.

Esempio: divisione per sottrazioni ripetute.

```
# Quante volte possiamo sottrarre 3 da 9?
n = 9
contatore = 0
while n > 0:      # CORRETTO: ci fermiamo quando n arriva a 0
    n = n - 3
    contatore = contatore + 1
print(contatore) # 3

while n >= 0:     # SBAGLIATO: il ciclo esegue ancora con n=0
    n = n - 3
    contatore = contatore + 1
print(contatore) # 4 - un'iterazione di troppo
```

Riepilogo degli errori frequenti

Errore	Sintomo	Come individuarlo
<code>j = j + 1</code> mancante	Ciclo interno non termina	Tracciare il valore di <code>j</code>
Flag reinizializzato dentro il ciclo	Il risultato viene sovrascritto a ogni iterazione	Verificare dove si trova l'inizializzazione del flag
Guardia troppo larga (<code>j < len</code> invece di <code>j < i</code>)	Si contano più elementi del dovuto	Tracciare quali coppie vengono effettivamente confrontate
Indentazione sbagliata	Il codice non esegue il flusso atteso	Controllare a quale blocco appartiene ogni riga
<code>>=</code> invece di <code>></code>	Risultato numerico sovrastimato di 1	Tracciare il valore della variabile all'ultima iterazione

6. Pattern: quale tipo di ciclo scegliere

Quando si scrive un ciclo `while` su una lista, si ricade quasi sempre in uno di questi due paradigmi:

Paradigma	Guardia	Quando usarlo
Scansione completa	<code>i < len(lista)</code>	devo esaminare <i>tutti</i> gli elementi (trasformare, filtrare, sommare...)
Ricerca con uscita anticipata	<code>i < len(lista)</code> <code>and not trovato</code>	mi interessa solo <i>trovare il primo</i> elemento che soddisfa una condizione

La scelta dipende dal problema: se devo raccogliere tutti i pari uso la scansione; se devo trovare solo il primo elemento positivo uso la ricerca anticipata.

Pattern: ricerca con uscita anticipata

Schema generale:

```
i = 0
trovato = False
posizione = -1

while i < len(lista) and not trovato:
    if lista[i] soddisfa_condizione:
        trovato = True
```

```

        posizione = i
    i = i + 1

```

Il ciclo continua finché `i` è ancora dentro la lista e non è stato ancora trovato alcun risultato. Quando si incontra un elemento che soddisfa la condizione, si imposta `trovato = True` e si salva la posizione. Alla valutazione successiva il ciclo si interrompe.

Se nessun elemento soddisfa la condizione, `posizione` mantiene il valore `-1`, che segnala l'assenza di un risultato valido.

Esempio: primo elemento positivo

```

def primo_positivo(lista):
    trovato = False
    posizione = -1
    i = 0
    while i < len(lista) and not trovato:
        if lista[i] > 0:
            trovato = True
            posizione = i
        i = i + 1
    return posizione

print(primo_positivo([-3, -1, 5, 2]))    # 2
print(primo_positivo([-3, -1, -5]))     # -1

```

7. Costruire una nuova lista con una funzione pura

Un pattern ricorrente è costruire una nuova lista modificata senza toccare quella originale. La struttura è sempre la stessa:

1. creare una lista risultato vuota;
2. scorrere la lista originale con un ciclo;
3. per ogni elemento, decidere cosa aggiungere alla lista risultato;
4. restituire la lista risultato con `return`.

Esempio step-by-step: inserire `k` dopo ogni numero pari

Scrivere una funzione pura che riceve una lista di interi e un intero `k`, e restituisce una nuova lista in cui `k` compare dopo ogni numero pari della lista originale.

```

lista = [3, 4, 9, 2, 8, 7]    k = 0
risultato atteso: [3, 4, 0, 9, 2, 0, 8, 0, 7]

```

Step 1 — firma della funzione

```

def inserisci_dopo_pari(lista, k):
    ...
    return risultato

```

Step 2 — dove metto i risultati?

Non posso modificare `lista`. Ho bisogno di una lista nuova, inizialmente vuota.

```
def inserisci_dopo_pari(lista, k):
    risultato = []
    ...
    return risultato
```

Step 3 — devo scorrere tutta la lista (ogni pari potrebbe apparire anche alla fine): uso la **scansione completa**.

Step 4 — cosa faccio per ogni elemento?

Ogni elemento va aggiunto sempre a `risultato`. Se è pari, aggiungo anche `k` subito dopo.

```
def inserisci_dopo_pari(lista, k):
    risultato = []
    i = 0
    while i < len(lista):
        risultato.append(lista[i])
        if lista[i] % 2 == 0:
            risultato.append(k)
        i = i + 1
    return risultato
```

Verifica:

i	lista[i]	pari?	risultato dopo il passo
0	3	No	[3]
1	4	Sì	[3, 4, 0]
2	9	No	[3, 4, 0, 9]
3	2	Sì	[3, 4, 0, 9, 2, 0]
4	8	Sì	[3, 4, 0, 9, 2, 0, 8, 0]
5	7	No	[3, 4, 0, 9, 2, 0, 8, 0, 7]

```
numeri = [3, 4, 9, 2, 8, 7]
nuovi = inserisci_dopo_pari(numeri, 0)
print(numeri)    # [3, 4, 9, 2, 8, 7] - invariata
print(nuovi)     # [3, 4, 0, 9, 2, 0, 8, 0, 7]
```

8. Il ciclo for

Da **while** a **for**

Abbiamo scritto molti cicli con questa struttura:

```
i = 0
while i < len(lista):
```

```
# fai qualcosa con lista[i]
i = i + 1
```

Questo schema ha tre parti meccaniche da ricordare ogni volta: inizializzare `i`, scrivere la guardia giusta, aggiornare `i`. Se dimentichiamo anche solo l'aggiornamento, il ciclo gira all'infinito.

Concettualmente, però, quello che vogliamo fare è spesso più semplice: **esaminare ogni elemento della lista in ordine**, senza curarci della sua posizione. Il ciclo `for` ci permette di scrivere la stessa cosa delegando a Python la gestione dell'indice:

```
for elemento in lista:
    # fai qualcosa con elemento
```

Le due versioni sono equivalenti:

<i># con while</i>	<i># con for</i>
i = 0	for elemento in lista:
while i < len(lista):	print(elemento)
print(lista[i])	
i = i + 1	

Sintassi

```
for variabile in espressione_iterabile:
    istruzioni
```

- `for` introduce l'iterazione;
- `variabile` è il nome che prende l'elemento corrente a ogni passo;
- `espressione_iterabile` è qualunque valore che può fornire i suoi elementi uno alla volta: liste, stringhe, e altri tipi che vedremo;
- i due punti `:` aprono il blocco, che va indentato.

Semantica

1. Python prende un elemento dall'iterabile.
2. Lo assegna alla variabile di iterazione.
3. Esegue il blocco.
4. Passa all'elemento successivo.
5. Quando gli elementi finiscono, il ciclo termina.

Tracciamento del `for`

```
for parola in ["Ciao", "Mondo", "Python"]:
    print(parola)
```

Passo	Codice sorgente	Stato	Output
1	<code>for parola in [...]</code>	<code>parola→"Ciao"</code>	
2	<code>print(parola)</code>	<code>parola→"Ciao"</code>	Ciao
3	<code>for parola in [...]</code>	<code>parola→"Mondo"</code>	
4	<code>print(parola)</code>	<code>parola→"Mondo"</code>	Mondo
5	<code>for parola in [...]</code>	<code>parola→"Python"</code>	
6	<code>print(parola)</code>	<code>parola→"Python"</code>	Python

for vs while: quando usare quale

Con il **while** si itera sull'**indice** e si recupera il valore come `lista[i]`: a ogni passo si hanno a disposizione sia la posizione che il valore.

Con il **for** si itera direttamente sul **valore**: la variabile di iterazione è già l'elemento, non la sua posizione.

```
while:    i → lista[i] → valore
for:      elemento → valore
```

Se il problema richiede la **posizione** dell'elemento (ad esempio: restituire l'indice del primo positivo), bisogna usare il **while**. Se interessa solo il **valore**, il **for** è più compatto e leggibile.

9. La funzione range()

`range` è un tipo iterabile e immutabile che produce una sequenza di interi. Non è una lista — non occupa spazio in memoria per tutti i valori — ma si può usare direttamente in un `for`.

```
range(n)          # da 0 a n-1
range(a, b)       # da a a b-1
range(a, b, s)    # da a a b-1, con passo s
```

Ripetere un'azione n volte

```
for i in range(5):
    print("ciao")    # stampa "ciao" 5 volte
```

Quando il valore della variabile di iterazione non serve, si usa per convenzione `_`:

```
for _ in range(5):
    print("ciao")    # "_" segnala che la variabile è inutilizzata
```

Esempi di sequenze

```
range(10)          # 0, 1, 2, ..., 9
range(7, 15)       # 7, 8, 9, ..., 14
range(0, 10, 2)    # 0, 2, 4, 6, 8    (passo 2 → numeri pari)
```

```
range(1, 10, 2)  # 1, 3, 5, 7, 9   (passo 2, partenza da 1 → dispari)
range(25, 35)   # 25, 26, ..., 34
```

Generare una lista di numeri pari

```
pari = []
for n in range(0, 10, 2):
    pari.append(n)

print(pari)  # [0, 2, 4, 6, 8]
```

10. Pattern con for

Con il `for` ricorrono tre schemi frequenti.

Scansione

Si esamina ogni elemento e si fa qualcosa con il suo valore.

```
for carattere in parola:
    print(carattere)
```

Trasformazione

Si costruisce una nuova lista applicando un'operazione a ogni elemento. Con il `for` non si può agire in-place (non si ha l'indice), quindi si costruisce sempre una lista nuova.

```
parole = ["Giacomo", "Alessia", "Martina"]
parole_minuscole = []

for parola in parole:
    parole_minuscole.append(parola.lower())
```

Accumulo

Si calcola un valore aggregato (somma, conteggio, massimo...) esaminando tutta la lista.

```
numeri = [4, 9, 10, 2, -3]
somma = 0

for numero in numeri:
    somma = somma + numero
```

11. Liste di liste

Una lista può contenere qualsiasi tipo di valore — incluse altre liste. Questo permette di rappresentare strutture a due dimensioni come **matrici**, tabelle e

griglie.

```
matrice = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

`matrice` è una lista di 3 elementi, ognuno dei quali è una lista di 3 interi.

Accesso agli elementi

Per accedere a un elemento servono due indici: prima la riga, poi la colonna.

```
matrice[0]      # [1, 2, 3] - prima riga intera  
matrice[0][0]   # 1       - riga 0, colonna 0  
matrice[1][2]   # 6       - riga 1, colonna 2  
matrice[2][-1]  # 9       - ultima colonna della terza riga
```

	col 0	col 1	col 2
riga 0	1	2	3
riga 1	4	5	6
riga 2	7	8	9

Per scorrere tutti gli elementi si usano cicli annidati: il ciclo esterno scorre le righe, quello interno le colonne.

```
for riga in matrice:  
    for elemento in riga:  
        print(elemento)
```